

C程错题集

低级错误

1. sum初始值不为0
2. int溢出
3. 嵌套/缩进
4. 输入输出格式
5. 若if/for/while后未加{}, 只包括紧接着的一条语句
6. 循环
7. 改过的答案切记马上保存
8. 英文状态输入

调试

1. 步骤
 - 用editplus写代码, 保存为abc.c,拖进编辑器
 - F9编译—F5启动调试—F8进入函数—F7单步执行—ctrl+F5运行到光标
 - 进入函数F8

基础

1. 先定义再赋值
2. 5 (x-9)/9
3. == 和=
4. 实型不可%
5. 定义的变量代表内存中一个存储单元
6. scanf输出格式, /' '
7. C源程序经编译和连接才可运行出结果
8. int初始化不能连等于
9. 不能用数字字符进行算数运算,判断要用== '1'
10. scanf不能输入带空格的字符串要用gets()
11. 有且只有一个main, 但可有多c.c文件包含.h文件)
12. 关键字不能用作变量名、函数名或其他标识符 (union,enum,void,extern

在编程和计算机科学中, "基类型" (Base Type) 通常指的是构成更复杂数据结构的基本数据类型。这些类型是不可再分的, 是构建其他数据类型的基础。在不同的编程语言中, 基类型可能有不同的名称和种类, 但它们通常包括以下几种:

1. **整型** (Integer) : 用于表示整数, 如 `int`、`short`、`long` 等。
2. **浮点型** (Floating Point) : 用于表示小数, 如 `float`、`double` 等。
3. **字符型** (Character) : 用于表示单个字符, 如 `char`。
4. **布尔型** (Boolean) : 用于表示逻辑值, 如 `bool` 或 `boolean`。
5. **指针型** (Pointer) : 用于存储内存地址, 如 `int*` 表示指向整数的指针。
13. 6. **空类型** (Void) : 表示没有类型, 通常用于函数返回类型, 表示函数不返回任何值。

Pasted image 20250103105324.png

14. 指针的基类型是它指向的数据的类型

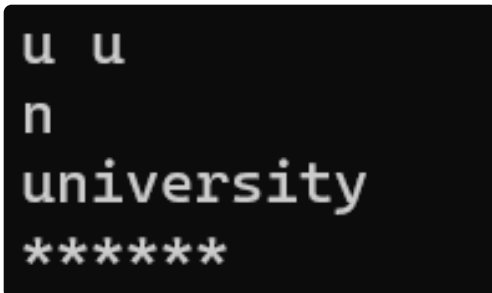
分支结构

1. 字符常量单字符单引号
2. 表达式的返回值和副作用
3. 检查分支语句是否正确至少用测试用例: `printf`个数
4. `inf`无穷
5. `case`常量表达式不能相同
6. 浮点值在计算机中储存为一串数字
7. `int`溢出-214783648到214783647
8. `switch`

请写出下列程序的运行结果：

```
#include <stdio.h>
int main( )
{
    int i;
    char str[]="university";
    for(i=0; i<=3; i++)
        switch(i) {
            case 0: printf("%c ",str[i]);
            case 1: printf("%c\n",str[i]);break;
            case 2: printf("%s\n",str);
            default: printf("***");
        }
    return 0;
}
```

Pasted image 20241225191617.png



Pasted image 20241225191635.png

循环结构

1. sum 没有附初值0
2. 死循环while(i<=10);
3. for循环的三个表达式都可以省略

函数

1. 参数传递单向
2. 动态（自动变量/形参）||静态（静态局部变量/全局变量）存储区
3. 凡在函数中未指定存储类别的局部变量，其默认的存储类别为auto,而非static

4. Initialisation is not mandatory for local static variables不需要显式初始化
5. 未赋初值：动态随机/静态0
6. 实参与形参必须
7. 个数相等，类型一致
8. 未显式说明，隐含类型int
9. `Func(exp1 , exp2+exp3 , exp4*exp5)` 实参3个
10. 定义`double fun(int x, int y)
11. break 语句只能用于循环体和switch语句体内
12. 局部静态变量 (local static variables) 只能在该函数内调用
13. 不能在形参表内赋初值
14. 求素数

```
int fun3(int m)
{ int i;
  if (m == 2 || m == 3) return 1;
  if (m < 2 || m % 2 == 0) return 0;
  for (i = 3; i < m; i = i + 2)
    if (m % i == 0) return 0;
  return 1;}
```

15. 判断高位到低位从小到大

```
int fun (int m)

{int cur_digit, old_digit = 10;
  if (m < 0){m = -m;}
  do{
    cur_digit = m % 10;
    if( cur_digit >= old_digit){
      return 0;
    }
    old_digit =cur_digit;
    m=m/10;
  }while (m != 0);
  return 1
}
```

16. 映射类输出用switch
17. 求Fabonacci数列第n项

```
int fib( int n ){
if(n==1){return 1;}
if(n==2){return 1;}
int a = 1;int b = 1;int res = 2;
for(int i=3;i<=n;i++){
int c = b;b = a + b;a = c;}
return b;}
```

18. ==拆解任务模块化== 验证哥德巴赫猜想

```
#include<stdio.h>
#include<math.h>
%判断素数
int check(int x){
int res = 1;
for(int i=2;i<=sqrt(x);i++){
if(x%i==0){res = 0;break;}}
return res;}
%分解
int getP(int N){
int res = 2;
for(int i=2;i<N/2;i++){
if(check(i)&&check(N-i)){
res = i;break;}}
return res;}
%输入输出
int main(){
int N;
scanf("%d",&N);
int p = getP(N);
printf("%d = %d + %d",N,p,N-p);
return 0;}
```

类似：判断完数函数+输出函数+主函数循环在范围内查找并输出

19. 乘法表：双循环嵌套

20. 金字塔等形状输出：分段

21. 超时：算幂自定义函数

数据类型和表达式

1. 计算机位数有限，任何数字类型都只能表示有限个数字
2. int占32位，是有符号的，最高位是符号位，因此能表示的数字范围是 $-2^{31} \sim 2^{31}-1$ 。
(与之对比，unsigned int是无符号的，能表示的数字范围是 $0 \sim 2^{32}-1$)
3. 字符的多种表示

```
%十六进制
char c='\x41';int i=0x41;c=(char)i;
%八进制
char c='\101';int i=0101;c=(char)i;
printf("十进制%d转八进制%o转十六进制%x);
```

==辨析'A'与"A"

4. ||与&&具有短路性质
5. 优先级与运算顺序
6. 八进制：0开头/包含0-7
7. 'A'与65等价，a=6+'A'
8. scanf对不同格式符的输入
 - %d/%f自动跳过前导空格和换行符，直到读入有效数字字符（输入02则只读取2），如果输入非法字符如'/'会读取失败，输入为空
 - %c会把空格和换行当作有效输入：解决方法：scanf("%c %c",&a,&b)或在两个scanf间加getchar()
 - %s不能输入带空格的字符串，要用gets(s)
 - 输入变量的地址值
9. int溢出的补码机制
10. static变量初始化只会执行一次
11. strlen算的是字符串有效长度，不包括'\0'

数组

1. 输出矩阵

```
for(i = 0; i < 3; i++){
    for(j = 0; j < 4; j++){
        printf("%4d", tab[i][j]);
        printf("\n");
    }
```

2. 定义时为长度非下标
3. 数组名做函数参数没有在函数中新建数组

4. 字符串本质为字符数组，C中数组不能整体赋值（因为数组名为常量且内存确定）也不能比大小（实际上比的是地址）

"Pasted image 20241128151041.png" could not be found.

5. 二维数组定义与赋初值

"Pasted image 20241128151608.png" could not be found.

可小于不可超过，可省行不可省列

`int x[][3] = {0};` 这里只提供了一个初始化值 `{0}`，这意味着我们只有一行，而且这一行的三个元素都被初始化为0。因此，编译器可以推断出第一维的大小至少为1（因为我们至少有一行）。由于我们没有提供更多的行，编译器会认为第一维的大小就是1。

6. 字符数组定义与赋初值（以下等价

```
static char s[6] = {'H', 'a', 'p', 'p', 'y', '\0'};
static char s[6] = {'H', 'a', 'p', 'p', 'y', 0};
static char s[6] = {'H', 'a', 'p', 'p', 'y'};
static char s[6] = "Happy";
static char s[6] = {"Happy"};
```

7. 字符串复制
8. 删除字符串中空格
9. 选择法排序：双循环找最大下标并交换
10. 矩阵处理
11. 找最大值
12. 输出杨辉三角
13. 二分查找!
14. 冒泡排序
15. 数组逆序
16. 环形报数
17. 凯撒密码
18. 去掉重复字符后从大到小排序后输出，可颠倒操作顺序

指针

1. 指针只有加减操作，没有乘除操作。指针可以加常数、减常数；两个同类型指针可以相减，其结果是以所指向类型大小为单位的差（不是地址值之差）；两个指针不可以相

加。

2. 直接访问就是直接利用变量的地址直接进行访问

3. 三个概念

- 野指针：引用前先对p赋值
- 空指针：p=0,0地址对应的内存单元不存放任何合法变量（所有整数中，只有0可以不强制类型转换直接赋值给指针变量）
- 通用指针：void *

void不能用来定义变量，void a语法错误a宽度未知

void可以表示函数返回值类型，此时无返回值

void可以表示形参，此时没有参数

1. 声明先判断在定义什么，函数/数组/指针;只有 (* p) 才是定义指针

2. 字符串输入输出时，指针未初始化，s未指向任何内存会读取错误

3. 指向int类型变量的指针的数据类型为int*

4. 字符串压缩要考虑重复字符大于10

5. 读最大长80的字符串，数组长至少81

6. &&x错误，为逻辑运算符，不能看作取两次地址

7. 定义一个指针数组并初始化赋值若干个字符串常量，则指针数组并不存放这些字符串，而仅仅指向各个字符串。(F)==指针数组存放指向字符串首元素地址的字符指针，而字符串是字符数组，指向字符串的是数组指针

8. 数组名可以被当作指针常量，不能被重新赋值，但array类型是 `int[10]` 是个变量，不是指针

▶ 1-5 分数 3

假设有定义如下： `int array[10];` 则该语句定义了一个数组array。其中array的类型是整型指针（即： `int *` ）。

☒ T

☐ F

评测结果 答案错误

得分 0 分

IQLPJxnrhWQR5znNAazNBvmwOffnf8TVfj4HW12crBiuAA_1785_428.png

本程序的功能是将字符串a的所有字符传送到字符串b中，要求每传送三个字符后再存放一个空格。

```
#include <stdio.h>
int main()
{
    int i,k=0;
    char a[80], b[80], *p;
    p=a;
    gets(p);
    while(*p)
    {
        for(i=1; *p!=0; p++, k++, i++) b[k]=*p;
        if(i%3==0) { b[k]=' '; k++; }
    }
    b[k]='\0';
    puts(b);
    return 0;
}
```

9.

Pasted image 20241211232905.png

结构

- 在对结构变量操作时，`.` 被称为成员或者分量运算符/在使用结构指针时，`->` 被称为指向运算符
- 递归调用算法与相同功能的非递归算法相比，主要问题在于重复计算太多，而且调用本身需要分配额外的空间、传递数据和控制，所以时间和空间开销通常都比较大。递归程序往往简洁易懂，但占用较大空间。递归层数过大会造成系统堆栈溢出。
 - 堆栈溢出是说**堆区和栈区的溢出**，二者同属于缓冲区溢出。从上面关于堆区和栈区的解释可以看出，一旦程序确定，堆栈内存空间的大小就是固定的，当数据已经把堆栈的空间占满时，再往里面存放数据就会超出容量，发生上溢；当堆栈中的已经没有数据时，再取数据就无法取到了，发生下溢。
- C语言中结构类型变量在程序执行期间，所有成员一直驻留在内存中
- 只有结构体变量才分配地址，而结构体的定义是不分配空间的==结构体是一种自定义的数据类型，是创建变量的模板，不占用内存空间；结构体变量才包含了实实在在的数据，需要内存空间来存储
- 结构体变量名和数组名不同，数组名在表达式中会被转换为数组指针，而结构体变量名不会，无论在任何表达式中它表示的都是整个集合本身，要想取得结构体变量的地址，必须在前面加&符号
- 结构体变量不能整体引用，只能引用变量成员
- 结构体成员是指针类型变量，以很好地防止内存被浪费，但是注意在引用时一定要给指针变量分配地址 `stu.name=(char*)malloc(sizeof(char));`//内存初始化
- 注意区分**使用typedef关键字定义结构体类型& 定义结构体类型的同时定义结构体类型变量

[【C语言】typedef的使用_ttypedef在c语言中用法-CSDN博客](#)

下面对typedef的叙述中错误的是（ ）。

- ☐ A. 用typedef可以定义各种类型名，但不能用来定义变量
- ☒ B. 用typedef可以增加新类型
- ☐ C. 用typedef只是将已存在的类型用一个新的标识符来代表
- ☐ D. 使用typedef有利于程序的通用和移植

Pasted image 20241227142446.png

指针进阶

1. 二级指针变量可以指向同类型的一级指针变量，不可以指向同类型的普通（非指针）变量
2. 指针函数返回0或NULL，表示指针不指向任何有效的内存地址
3. 指向整数指针的指针与指向整数数组的指针可以看成是等价的类型（均为二级指针
4. 函数返回值

- return 0; 一般0代表的是正常结束
- return -1; -1代表非正常结束
- return ; void 返回是空，void返回值可以什么也不写，可以写return;
- return NULL; 如果数据类型是指针类型，那我们一般成功的时候返回相应的地址，失败的时候返回NULL``(stdio.h头文件中有#define NULL 0;)

5. 数组名为常量，不可做为运算操作数；指针需要赋值有指向的位置才能调用
6. 指针做函数返回值可以返回0，相当于NULL（即空指针）。但是不能返回负数。因为指针的本质是地址，地址可以为0但不能为负数。

1-11. F，指向整数指针的指针与指向整数数组的指针可以相互转化（见 HW11 4-2），但是这两个类型是不同的。主要区别在于，指针的运算是基于指针指向的类型的的大小的，如下面的代码里，p 是指向 int，q 指向长度为 4 的 int 数组，因此 p++ 会使 p 指向下一个 int，而 q++ 会使 q 指向下一个长度为 4 的 int 数组。

```
int *p;           // 指向 int
int (*q)[4];      // 指向 int [4]
```

7.

Pasted image 20250102195709.png

8. 求子矩阵和：二维前缀和

9. 指针加减法/偏移及强制类型转化 [int_ptr1=\(int \)\(&a+1\)数组指针_int_ptr=\(int\)\(&a+1\)-CSDN博客](#)

10. 定义一个指针数组并初始化赋值若干个字符串常量，则指针数组并不存放这些字符串，而仅仅指向各个字符串
11. 对于存放结果的指针不能改变其地址，关联主调改变所指向的值
12. 在用数组名作函数参数传递数组时，下面两种函数声明没有区别（在函数声明中，数组参数会被隐式地转换为指向其首元素的指针）

①void fun(int arr[], int n);

②void fun(int *arr, int n);

Pasted image 20241213124738.png

13. 一维数组名做函数参数若数组大小不确定，需要函数内部另设一个n
14. 二维数组名做函数参数且行列数均不确定

- `` void print(int * a,int row,int col)
- 动态内存分配`void print(int ** a,int row,int col)
- 调用一次动态内存分配函数分配的内存空间是连续的

1. 命令行参数[C语言命令行参数详解-CSDN博客](#)

2. 在命令行中运行一个程序时，需要提供的是编译后的可执行文件的名称，而不是源代码文件的名称。该可执行文件在 Unix-like 系统中通常没有特定的后缀，在 Windows 系统中通常以 `.exe` 结尾。

3. `argc` 程序名称也算一个

4. `argv` 数组包含以下内容：`argv` 指针指向的数组的大小是参数的数量加1

`argv[0]`：指向程序的名称（或者在某些系统上是第一个参数）。

`argv[1]` 到 `argv[n-1]`：指向传递给程序的参数，其中 `n` 是参数的数量。

`argv[n]`：总是一个空指针（`NULL`），表示参数列表的结束。

5. `argv[argc]` 是一个指向 `NULL` 的指针。

当你尝试执行 `printf("%s\n", argv[argc]);` 时，你实际上是在尝试打印一个空指针指向的字符串。在C语言中，尝试解引用一个空指针（即尝试访问空指针指向的内存）是未定义行为，会产生一个段错误（段错误是因为程序试图访问它没有权限访问的内存区域，操作系统通过中断来阻止这种非法操作，以保护系统的稳定性。

What will be the output of the following C statement? (assuming the input is “cool brother in city”)

```
printf(“%s\n”, argv[argc]);
```

- ☐ A. (null)
- ☐ B. City
- ☐ C. In
- ☒ D. Segmentation Fault

Pasted image 20241227143650.png

6. 命令行参数在命令提示符cmd中输入，文件名称前默认有根目录

以下程序经过编译连接后的可执行文件名是How.exe，执行How are you!<回车>，输出结果是_____.

```
#include<stdio.h>
```

```
#include<string.h>
```

```
int main( int argc, char *argv[])
```

```
{    int len=0;
```

```
    printf("%c, ",*(*(argv+1)+1)+1);
```

```
    while (--argc > 0)
```

```
        len += strlen( argv[argc]);
```

```
    printf("%d", len);
```

```
}
```

Pasted image 20241227161947.png

输出: s,7

下面程序的运行结果是

8

1分。

```
#include<stdio.h>
int main(void)
{
    int i, j, sum, *p[4];
    static int a[4][4];

    for(i = 0; i < 4; i++){
        p[i] = &a[i][0];
    }
    for(i = 0 ; i < 4 ; i++) {
        *(p[i] + i) = 1 ;
        *(p[i] + 4 - (i + 1)) = 1;
    }
    sum = 0;
    for(i = 0; i < 4; i++){
        for(j = 0; j < 4; j++){
            sum += p[i][j];
        }
    }
    printf("%d", sum);

    return 0;
}
```

7.

Pasted image 20250102185053.png

执行以下程序，第1行输出

10 20 10 20 10 20

1分

；最后1行输出

20 10 10 20 20 10

1分

。

```
/* 二级指针操作示例 */
#include<stdio.h>
int main(void)
{
    int a, b, t;
    int *pa = &a, *pb = &b, *pt;
    int **ppa = &pa, **ppb = &pb, **ppt;

    a = 10, b = 20;
    printf("%d %d %d %d %d %d\n", a, b, *pa, *pb, **ppa, **ppb); /* 输出数据之间间隔一个空格 */
    ppt = ppb; ppb = ppa; ppa = ppt;
    printf("%d %d %d %d %d %d\n", a, b, *pa, *pb, **ppa, **ppb); /* 输出数据之间间隔一个空格 */
    pt = pb; pb = pa; pa = pt;
    printf("%d %d %d %d %d %d\n", a, b, *pa, *pb, **ppa, **ppb); /* 输出数据之间间隔一个空格 */
    t = b; b = a; a = t;
    printf("%d %d %d %d %d %d\n", a, b, *pa, *pb, **ppa, **ppb); /* 输出数据之间间隔一个空格 */

    return 0;
}
```

8.

Pasted image 20250102194205.png

4-2. 注意这里 &a 的类型是 `int (*)[4]` 而不是 `int **` (因为 `a` 是一个数组而不是 `int *` 指针)，即指向长度为 4 数组的指针。所以 `&a + 1` 相当于指向 `a` 之后的下一个数组 (这里假设为 `b`)。随后我们将 `&a + 1` 这个指针进行类型转化，转化为 `int *` 指针，指向的地址不变 (还是 `b`)，但是现在指向的是 `b` 里的第一个元素 (而非整个数组)。最后 `ptr1[-1]` 相当于 `*(ptr1-1)` 即 `*b` 的前一个 `int` 元素，也就是数组 `a` 的最后一个元素，为 4。

```
int a[4] = { 1, 2, 3, 4 }; // 假设 a[4] 后面紧挨下一个数组 b[4] =
int *ptr1 = (int *)(&a + 1);
printf( "%x", ptr1[-1]);
```

9.

Pasted image 20250102195139.png

2-2. C 错误的地方在于，`&x` 是一个地址，但是这个地址并没有被实际存在内存里或者放在某个变量里，因此没有对应的地址存 `&x`，所以不能写 `&&x`。相反如果使用下面的写法就是正确的，因为此时 `&x` 的值被放在了一个指针变量 `p` 里。

```
int *p = &x;
int **q = &p; // correct
```

10.

Pasted image 20250102200100.png

2-9. 2-11. 假设 `int *p` , 区分: `*p++`, `*++p`, `(*p)++`

- `*p++` : 先取 `p` 指向的值, 然后 `p` 指向下一个元素。
- `*++p` : 先 `p` 指向下一个元素, 然后取 `p` 指向的值。
- `(*p)++` : 先取 `p` 指向的值, 然后对这个值加 1。

```
int a[4] = { 1, 2, 3, 4 };
int *p = a;
printf("%d\n", *p++);    // a[0]=1, p 指向 a[1]=2
printf("%d\n", *++p);    // a[2]=3, p 指向 a[2]=3
printf("%d\n", (*p)++);  // a[2]+1=4, p 指向 a[2]=4
```

11.

Pasted image 20250102200527.png

For definitions: `char s[2][3]={ "ab", "cd" }, *p=(char *)s;` the expression `__` is correct and its value is equivalent to the element `s[1][1]`.

- ☐ A. `*(s+3)`
- ☐ B. `*s+2`
- ☐ C. `p[1][1]`

12.

- ☒ D. `*++p+2`

Pasted image 20250102200741.png

题目要求调用input函数输入N个字符串。

```
#include <stdio.h>
#define N 5
void input(char **p, int n);
int main(void)
{
    char data[N][81], *ptr[N];
    int i;
    for(i=0;i<N;i++)
        *(ptr+i)=*(data+i);
    input(ptr,N);
    for(i=0;i<N;i++)
        printf("%s\n",*(ptr+i));
    return 0;
}
void input(char **p, int n)
{
    int i;
    for(i=0;i<n;i++)
        scanf("%s",*(p+i));
}
```

1分

2分

13.

Pasted image 20250102202325.png

下面程序运行后的输出结果是____。

```
1  #include <stdio.h>
2  #include <string.h>
3  int main( )
4  {
5      char str[ ][20]={"Hello","Beijing"},*p=str[0];
6      printf("%d\n",strlen(p+20));
7      return 0;
8  }
```

- ☐ A. 0
- ☐ B. 5
- ☒ C. 7
- ☐ D. 20

14.

Pasted image 20250103103439.png

函数与程序结构

[深刻理解递归——从底层存储（栈图）来理解 - 知乎 \(zhihu.com\)](#)

[递归的压栈出栈过程 - 动图演示_递归压栈过程-CSDN博客](#)

1. 二分求幂

本题要求采用递归方式求x的y次方。其中x为正整数，y为非负整数。

```
///x^y
int getPower(int x, int y) {
    if (y == 0)
        return 1;

    int res = 

|                    |     |
|--------------------|-----|
| getPower(x, y / 2) | 1 分 |
|--------------------|-----|

;
    if (y % 2 == 0)
        return res * res;
    else
        return 

|               |     |
|---------------|-----|
| x * res * res | 1 分 |
|---------------|-----|

;
}
```

Pasted image 20241211212850.png

```
int getSum(int k){
    int s;
    s = getSum(k+1) + k;
    return s;
}

int main(){
    int sum;
    sum = getSum(0);
    return 0;
}
```

递归函数 `getSum` 设计不合理，这段程序将无限递归，始终不能结束。

☐ T ☒ F

2.

当栈溢出后递归终止

- 递归函数存在堆栈空间过度使用的风险，不能无限进行
- 递归调用算法与相同功能的非递归算法相比，主要问题在于重复计算太多，而且调用本身需要分配额外的空间、传递数据和控制，所以时间和空间开销通常都比较大。
- 递归并不节省存储器的开销，因为递归调用过程中必须在某个地方维护一个存储处理值的栈。
- 递归的执行速度并不快，但递归代码比较紧凑，并且比相应的非递归代码更易于编写与理解。
- 函数递归调用自身时，每次调用都会得到一个与以前的变量集合不同的新的变量集合。

8. 累加求和

```
int f(int n) {
    if (n) return f(n - 1) + n;
    else return n; }
```

9. 求x的y次方

```
#include<stdio.h>
void main()
{
    float fun(float,int);
    float a;
    int n;
    scanf("%f",&a);
    scanf("%d",&n);
    printf("%.3f",fun(a,n));
}
float fun(float x,int y)
{
    int i=1;
    float fz=1;
    if(y==0)
        return 1;
    while(i<=y)
        {fz *=x;i++;} return fz;}
```

4. 求n的阶乘

```
int JC (int n)
{
    if (n == 1)
        return 1;
    else
        return n * JC(n - 1);           //阶乘的递归实现方式 }
```

5. strlen()

```
int my_strlen(char* str)
{if (*str != '\0')
```

```

{
    return 1 + my_strlen(str + 1);    //strlen函数的模拟实现方式
}
return 0;}

```

6. 字符串求逆

```

void reverse_string(char* string)
{
    int len = strlen(string);
    char tmp = *string;
    *string = *(string + len - 1);
    *(string + len - 1) = '\0';
    if(strlen(string+1) > 1 )
        reverse_string(string + 1);
    *(string + len - 1) = tmp;    //这一步才能赋值，把tmp 赋值为'\0'}
}

```

7. 将一个函数说明为static后，只能被同一源文件中的函数调用，不能被其他源文件中的函数调用

多文件程序

在多文件程序中，编译器会对每个 `.c` 文件单独编译。

1. **单独编译**：每个 `.c` 文件被单独编译成对应的目标文件（`.o` 文件或 `.obj` 文件），这个过程通常由编译器自动完成。
2. **编译器处理**：编译器处理 `.c` 文件，检查语法错误，进行优化，并生成机器代码。这个机器代码还不是最终的可执行代码，而是需要进一步链接的中间代码。
3. **链接**：所有单独编译的目标文件随后会被链接器（Linker）链接在一起，同时可能还会链接其他的库文件（如数学库、标准输入输出库等），最终生成一个单一的可执行文件。
4. **头文件**：`.h` 文件（头文件）在这个过程中起到声明和定义接口的作用，它们被包含在 `.c` 文件中，以确保编译器知道如何正确地处理函数调用和变量访问。
5. **依赖管理**：在大型项目中，可能需要管理文件之间的依赖关系，确保在编译过程中，依赖的文件被正确地更新和重新编译。
6. **编译命令**：在命令行中，可以使用如 `gcc` 或 `clang` 等编译器，通过指定多个 `.c` 文件作为参数来编译整个项目。例如：

```
gcc -o program file1.c file2.c file3.c
```

这个命令会编译 `file1.c`、`file2.c` 和 `file3.c`，并链接它们以及可能的其他库文件，最终生成名为 `program` 的可执行文件。

在一个源文件中定义的全局变量，其作用域为从定义该变量的位置开始至本文件结束。如果需要在多个文件间共享全局变量，通常需要使用头文件和 `extern` 关键字来声明这些变量。

一个工程包含以下几个文件，程序执行输出为（）。

```
//main.c
extern int num;
extern int calculate1();
extern int calculate2();
int main() {
    printf("%d %d %d",num,calculate1(),calculate2());
    return 0;
}
```

```
//source1.c
static int num=5;
int calculate1() {
    return num;
}
```

```
//source2.c
int num=3;
int calculate2() {
    return num;
}
```

☐ A. 5 3 5

☒ B. 3 5 3

Pasted image 20241227173341.png

如果 `source1.c` 和 `source2.c` 都定义了全局变量 `num`，并且没有使用 `static` 关键字，那么 `main.c` 中的 `extern int num;` 将指向先遇到的一个定义。如果编译器先处理 `source1.c`，那么 `extern int num;` 将指向 `source1.c` 中的 `num`。

下列叙述中错误的是()。

- ☐ A. C语言源程序经编译后生成扩展名为obj的目标程序

a.cn/problem-sets/1872214243227865088/paper/type/answer

PTA | 程序设计类实验辅助教学平台

- ☐ B. C程序经过编译、连接步骤之后才能形成一个真正可执行的二进制机器指令文件
- ☐ C. 用C语言编写的程序称为源程序,它以ASCII 代码形式存放在一个文本文件中
- ☒ D. C语言中的每条可执行语句和非执行语句最终都将被转换成二进制的机器指令

Pasted image 20250103100302.png

可执行语句：赋值/函数调用/控制流/表达式

非执行语句(不会转二进制)：声明/注释

预处理

[C语言预处理指令详解：宏定义、文件包含与条件编译-CSDN博客](#)

1. C语言中的预处理命令都以“#”开头
2. 预处理在对源程序中其他语句正式编译之前进行的
3. 宏名不是必须用大写字母表示，满足标识符命名规则即可。但替换文本可以包含任何字符。
4. 在头文件的最开始加入 `#pragma once` 就能够保证头文件只被编译一次

以下正确的叙述是（ ）。

- ☒ A. 宏替换不占用运行时间，只占用编译时间
- ☐ B. 在程序的一行中可以出现多个有效的预处理命令行
- ☐ C. 使用带参数宏定义时，参数的类型应与宏定义时一致
- ☐ D. 宏定义不能出现在函数内部

5.

6. 字符串中替换用 # 形参

执行以下代码段的结果为：

t=7

2 分

```
#define MAX(A,B) (A)>(B)?(A):(B)
#define PRINT(Y) printf("#Y"="%d\n", (Y));

int a=1,b=2,c=3,d=4;
int t=MAX(a+b,c+d);
PRINT(t);
```

下列选项中不会引起二义性的宏定义是()。

- ☐ A. `#define POWER(x) x*x`
- ☐ B. `#define POWER(x) (x)*(x)`
- ☐ C. `#define POWER(x) (x*x)`
- ☒ D. `#define POWER(x) ((x)*(x))`

7.

在“文件包含”预处理语句的使用过程中，当#include后面的文件名用双引号括起来时，寻找被包含文件的方式是()。

- ☐ A. 直接按系统设定的标准方式搜索目录
- ☒ B. 先在源程序所在目录搜索，再按系统设定的标准方式搜索

8.

请完成以下多文件编程题。程序需要实现计算两个整数的和与差，并将结果输出到控制台。

文件1: main.c

```
#include <stdio.h>
// 包含头文件
#include<calculation.h> | #include"calculation.h" | #inc

// 引用全局变量num1和num2
extern int | int 3分 num1;
extern int | int 3分 num2;
int main() {
    num1 = 10;
    num2 = 5;
    int sum = add(num1, num2);
    int difference = subtract(num1, num2);
    printf("%d,%d",sum,difference);
    return 0;
}
```

文件2: calculation.h

```
// 头文件保护，避免calculation.h被重复包含
#ifndef _CALCULATION_H 3分
#define _CALCULATION_H 3分
int add(int a, int b);
int subtract(int a, int b);
#endif 3分 // _CALCULATION_H
```

文件3: calculation.c

```
#include<calculation.h> | #include"calculation.h" | #inc

int add(int a, int b) {
    return a + b;
}
```

intia.cn/problem-sets/1872214243227865088/paper/type/answer

27

PTA | 程序设计类实验辅助教学平台

```
}
int subtract(int a, int b) {
    return a - b;
}
```

9.

Pasted image 20250103110510.png

链表

1. 插入和删除多：链表（在数组中插入或删除元素通常需要移动元素来维持数组的连续性）；查询多：数组（数组中元素可通过索引访问，链表不支持随机访问，查询特定元素需要从头节点开始遍历链表，直到找到目标元素）
2. 单向链表只能向前遍历，因此元素必须按照插入顺序访问；双向链表可以向前、向后遍历，因此可以从链表中的任意节点开始，向前或向后访问其他节点
3. 单链表的结点结构Node包含数据域data、指针域next
4. 当前指针为 p ，则使 p 指向下一个结点的语句是`p=p->next`;
5. 首位闭合

```
#include <stdio.h>
struct HAR{ int x, y; struct HAR *p; } h[2];
int main(void) {
    h[0].x = 1; h[0].y = 2; h[1].x = 3; h[1].y = 4;
    h[0].p = h[1].p = h;
    printf("%d%d\n", (h[0].p)->x, (h[1].p)->y);
    return 0; }
```

6. 求单链表值最大的结点

```
int getMaxNode(LinkNode* head)
{
    if (head == NULL)
        return INT_MIN;
    int first = head->data;
    int m = getMaxNode(head->next) 4分;
    if (m > first) return m;
    else return first;
}
```

Pasted image 20241227164015.png

- 7.

文件

■ 工程文件

- 开发一款软件，一般需要编写很多代码文件（**源程序**），在生成应用程序时，需要将这些文件“告知”编译器
- 在集成开发环境（例如VS或Dev-C）进行工程/项目的创建，通过工程文件，来管理这些源程序。

■ 工程文件的作用

- **组织所用到的源程序**
- **#include（文件包含）的目录**
 - 当编译器处理文件包含的时候（#include “xxxx.h”），到哪里去找它们
- **管理链接时要用到的第三方库文件**

1.

ef5ff031c84f9b22825b8ff9ba2e17a.jpg

2. `printf("%d %d %d",NULL, '\0', EOF) >>0 0 -1`

3. 缓冲文件系统的文件缓冲区位于内存数据区

4. *fp 指向文件类型结构

5. "a+"打开，位置指针移到文件末尾

6. 根据数据的组织形式，数据文件被分为文本文件或者二进制文件

- 数据在内存中以二进制的形式存储，如果不加转换的输出到外存（磁盘文件等等），就是二进制文件。
- 如果要求在外存上以ASCII码的形式存储，则需要在存储前转换。以ASCII字符的形式存储的文件就是文本文件。
- 数据在内存中字符一律以ASCII形式存储，数值型数据既可以用ASCII形式存储，也可以使用二进制形式存储。
- 拿10000这个数据来举例，用ASCII码来存储需要5个字节（因为10000是五位数，每一位数都要用一个字节的ASCII码来表示），而用二进制存储只需要四个字节（一个整型大小为4字节）。

7. 文件指针指向文件缓冲区中文件数据的存取位置

8. 文件的读函数是从输入文件中读取信息，并存放在内存中

9. 文件由数据流形式组成，可按数据的存放形式分为二进制文件和文本文件

C语言处理文件的方法是将文件以数据流的形式处理，不是将文件作为一个结构集合来处理，所以，C语言中文件的组织方式是无结构的字符序列方式。

10. 只有通过文件指针变量才能调用相应文件，定义文件指针变量时，FILE必须大写

11. 文件可以顺序存取，也可以随机存取（通过文件指针定位到文件的任何位置，并从那里开始读取或写入数据

12. 计算机存储

- 内存暂时（断电数据丢失），外存长期
 - 外存>磁盘(电脑内置)>硬盘（Hard Disk Drive，简称HDD）和固态硬盘（Solid State Drive，简称SSD）
 - 文件存储在磁盘==相当于终端
 - 内存分布区域：代码区（二进制），内存静态分配{ 静态存储区（全局/静态局部），栈区（自动变量）}，堆区（内存动态分配）
14. 随机读写方式通常不用于文本文件。C语言中，为什么随机读写方式不适用于文本文件？C语言中随机读写方式不适用于文本文件-CSDN博客==随机操作只适用于二进制文件是错误的，两种文件都适用
 15. **文件的打开方式**可以是文本模式或二进制模式。文本模式通常用于文本文件，而二进制模式适用于所有类型的文件，包括文本文件和二进制文件。在二进制模式下，文件内容按字节读取，不会进行任何特殊处理。在文本模式下，可能会执行换行符转换等操作，这取决于操作系统。
 16. **文件扩展名**只是一个方便用户识别文件类型的约定，而操作系统和程序通常不依赖扩展名来确定文件的处理方式。无论是文本文件还是二进制文件，都可以使用 `.txt` 作为扩展名。关键在于文件的内容和程序如何读取和写入这些内容。
 17. 一般不能用任何一个文本编辑器打开二进制文件进行阅读，若打开会得到乱码（因为文本编辑器会尝试将字节解释为字符，而大多数字节并不对应于可打印的字符。）
 18. 文件指针不变，位置指针随读写操作变

关于文件指针，以下说法错误的是（ ）。

- ☐ A. 只有通过文件指针变量才能调用相应文件。
 - ☐ B. 定义文件指针变量时，FILE必须大写。
 - ☐ C. 一个文件指针变量只能对应一个文件。
 - ☒ D. 一个文件指针变量可以同时对应多个文件。
- 19.

Pasted image 20250103092123.png

20. 文件指针指向文件缓冲区中文件数据的存取位置 (F)文件指针是指向一个文件的指针，确切的是存放了用文件这个结构体所定义的对象的首地址，文件指针的移动是指在文件之间来移动。即文件指针指向那个结构体，结构体里的缓冲区存放文件数据

C语言中，在成功打开一个文件后，可以使用

文件指针

1 分

来获取文件缓冲

21. 区的FILE结构信息。

Pasted image 20250103105256.png

文件操作函数

[C语言文件操作超详解（万字解读，细致入微）-CSDN博客](#)

1.开关

- `FILE *fopen(const char *filename, const char *mode);` 所有操作前先打开

在文件成功打开后返回此文件信息区的起始地址，打开失败则返回一个NULL（空指针）

- `int fclose(FILE *stream);`

关闭成功就返回0值，否则返回EOF（-1）值

2.顺序读写

字符

- `int fgetc(FILE *stream);`

成功返回从文件中读取的字符的ASCII值，反之则会返回EOF（-1）值

- `int fputc(int c, FILE *stream);`

成功返回存入文件的字符的ASCII码值，否则返回EOF（-1）值

字符串

- `char *fgets(char *str, int num (str最大字符数包括'\0'), FILE *stream);`

从当前文件位置指针开始读取字符，直到遇到换行符（'\n'）或者读取到指定数量的字符之前，或者遇到文件结束标志（EOF）

成功后，该函数返回 *str* 的头指针；如果在尝试读取字符时遇到文件结尾，则会设置 *eof* 指示符。如果在读取任何字符之前发生这种情况，则返回的指针为空指针（并且 *str* 的内容保持不变）。

如果发生读取错误，则设置错误指示器`ferror`,并返回空指针（但 `str` 所指向的内容可能已更改）。

- `int fputs(const char str, FILE stream);`

成功返回一个非负值，否则返回EOF (-1) 值

数据块

- `size_t fread(void ptr, size_t size, size_t numb, FILE stream);`

成功返回读取数据的次数，否则返回0值

- `size_t fwrite(const void ptr, size_t size, size_t numb, FILE stream);`

成功返回输出的数据总数，如果此数字与 `count` 参数不同，则写入错误会阻止函数完成

格式化

- `fscanf(pf, "%d %s %s", &(s.age), s.name, s.sex);`

运行成功后，该函数返回所读取数据的个数（甚至为零），如果在读取时发生读取错误或到达文件末尾，则会设置正确的指示器；如果在成功读取任何数据之前发生任何一种情况，则返回 EOT

- `fprintf(pf, "%d %s %s", L.age, L.name, L.sex);`

成功运行时返回所输出的字节总数，否则返回一个负数并报错

3. 随机读写（适用于二进制文件

- `void rewind(FILE *stream);`
- `int fseek(FILE *stream, long int offset, int origin);`

功能：改变读写位置，文件的随机读写和顺序读写

origin:设置传入的stream指针的起始位置（可传入3种参数：SEEK_SET（设置指针指向文件开头）、SEEK_CUR（不改变指针位置）、SEEK_END（设置指针指向文件结束位置））

成功返回0,否则回非零值

如果发生读取或写入错误, 则设置错误指示器 (ferror)

- `long int ftell(FILE *stream);`

成功后, 返回位置指示器的当前值。失败时, 返回 -1L, 并将 `errno` 设置为系统特定的正值

4. 文件错误处理

- `int feof(FILE *stream);`

遇到文件末尾而结束读取时返回非0值, 其他原因未读到文件末尾返回0

- `int ferror(FILE *stream);`